



## B. Tech Degree Internal Assessment Test – I, August 2025

### CST 305 SYSTEM SOFTWARE - Scheme

Time: 1.5 hrs

Maximum Marks: 50

#### PART A

(Answer all questions)

1. Differentiate system software and application software with proper examples. 3

Any 3 difference with proper example – 3 marks

| System Software                                                  | Application Software                                           |
|------------------------------------------------------------------|----------------------------------------------------------------|
| Used for operating computer hardware                             | Used by user to perform a particular task                      |
| System dependent                                                 | System independent                                             |
| Installs when OS installs                                        | According to user requirements                                 |
| User does not interact with SS <u>bcz</u> it works in background | User interacts with AS                                         |
| Can run independently. Provides platform for running AS.         | Can't run independently. Can't run without the presence of SS. |
| OS, assembler, loader, linker, compiler...                       | Microsoft office package, media player, browsers...            |

2. Examine any three assembler directives used in SIC. 3

One mark for each assembler directive – 3 marks

- **START:** specify *name* and *starting address of the program*
- **END:** specify *end of program* and (option) *the first executable instruction in the program*
  - If not specified, use the address of the first executable instruction
- **BYTE:** direct the assembler to *generate constants*
- **WORD**
- **RESB:** : instruct the assembler to *reserve memory location* without generating data values
- **RESW**

3. Explain the format of the object program generated by a two-pass SIC Assembler, highlighting the contents of each record type. 3

Correct format of Header, Text and End record – 1mark each

- **Header record:**
  - Col. 1 H
  - Col. 2-7 Program name
  - Col. 8-13 Starting address (hex)
  - Col. 14-19 Length of object program in bytes (hex)
- **Text record**
  - Col.1 T
  - Col.2-7 Starting address in this record (hex)
  - Col. 8-9 Length of object code in this record in bytes (hex)
  - Col. 10-69 Object code (hex) (2 columns per byte)
- **End record**
  - Col.1 E
  - Col.2-7 Address of first executable instruction (hex)  
(END program\_name)

4. Write SIC instructions to swap the values of ALPHA and BETA.

3

*Any program with correct logic- 3 marks*

```
LDA ALPHA
STA GAMMA
LDA BETA
STA ALPHA
LDA GAMMA
STA BETA
:
:
ALPHA RESW 1
BETA RESW 1
GAMMA RESW 1
```

5. Develop the object code for the following instructions indicating the instruction formats used, assuming the value of RDREC is 1036.

3

```
RMO S, A
+JSUB RDREC
LDA #1
```

| Opcode | Machine code |
|--------|--------------|
| RMO    | AC           |
| JSUB   | 48           |
| LDA    | 00           |

*One mark for each instruction*

**RMO S, A** → **AC40**

**+JSUB RDREC** → **4B101036**  
 Address field → RDREC address ie, 01036  
 Opcode, JSUB → 48  
 n i x b p e = 1 1 0 0 0 1  
 (0100 1000)opcode (110001)nixbpe (01036)address

**LDA #1** → **010001**  
 Opcode, LDA → 00  
 n i x b p e = 0 1 0 0 0 0  
 immediate addressing, i=1  
 So (0000 0000)opcode (010000)nixbpe (001)disp

## PART B

*(Answer all questions)*

- 6 a) Explain in detail any 4 types of system software. 6

*Proper explanation of any four system software listed - 1.5 marks each*

*Assembler  
Linker  
Loader  
Macro Processor  
Text Editor  
Debugger  
Device Driver  
Compiler  
Interpreter  
Operating System*

- b) Examine the architecture of the SIC/XE machine. 8

*Memory - 1 mark  
Register – 1 mark  
Data Formats – 1 mark  
Instruction Formats – 2 marks  
Addressing Modes – 1 mark  
Instruction Set – 1 mark  
Input and Output – 1 mark*

- 7.a) Construct a sequence of SIC/XE instructions to set  $ALPHA = 4 * BETA - 9$ , assuming ALPHA and BETA are defined and immediate addressing is used for constants. 5

*Any program with correct logic- 5 marks*

```
LDA BETA
LDS #4
MULR S,A
SUB #9
STA ALPHA
:
:
ALPHA RESW 1
BETA RESW 1
```

- b) Write the pass one of a two pass assembler algorithm and different data structures used in it. 9

*Correct Algorithm – 7 marks  
Data structures (Symtab, Optab, Locctr) – 2 marks*

Pass 1:

```
begin
  read first input line
  if OPCODE = 'START' then
    begin
      save #[OPERAND] as starting address
      initialize LOCCTR to starting address
      write line to intermediate file
      read next input line
    end (if START)
  else
    initialize LOCCTR to 0
  while OPCODE ≠ 'END' do
    begin
      if this is not a comment line then
        begin
          if there is a symbol in the LABEL field then
            begin
              search SYMTAB for LABEL
              if found then
                set error flag (duplicate symbol)
              else
                insert (LABEL,LOCCTR) into SYMTAB
              end (if symbol)
            search OPTAB for OPCODE
            if found then
              add 3 (instruction length) to LOCCTR
            else if OPCODE = 'WORD' then
              add 3 to LOCCTR
            -----
            else if OPCODE = 'RESW' then
              add 3 * #[OPERAND] to LOCCTR
            else if OPCODE = 'RESE' then
              add #[OPERAND] to LOCCTR
            else if OPCODE = 'BYTE' then
              begin
                find length of constant in bytes
                add length to LOCCTR
              end (if BYTE)
            else
              set error flag (invalid operation code)
            end (if not a comment)
          write line to intermediate file
          read next input line
        end (while not END)
      write last line to intermediate file
      save (LOCCTR - starting address) as program length
    end {Pass 1}
```

#### OPTAB:

It is used to lookup mnemonic operation codes and translates them to their machine language equivalents. In pass 1 the OPTAB is used to look up and validate the operation code in the source program. In pass 2, it is used to translate the operation codes to machine language. OPTAB is organized as a hash table, with mnemonic operation code as the key.

#### SYMTAB:

This table includes the name and value for each label in the source program, together with flags to indicate the error conditions. During Pass 1 labels are entered into the symbol table along with their assigned address value as they are encountered. During Pass 2,

symbols used as operands are looked up the symbol table to obtain the address value to be inserted in the assembled instructions. SYMTAB is organized as a hash table.

#### LOCCTR:

LOCCTR is initialized to the beginning address mentioned in the START statement of the program. After each statement is processed, the length of the assembled instruction is added to the LOCCTR to make it point to the next instruction.

8. Develop object code and various data structures and variables for the following code.

7

| Location | Label | Opcode | Operand |
|----------|-------|--------|---------|
| 4000     | TEST  | START  | 4000    |
| ....     | FIRST | LDA    | FIVE    |
| ....     |       | STA    | ALPHA   |
| ....     | ALPHA | RESW   | 2       |
| ....     | FIVE  | WORD   | 5       |

*Correct Address and Object Code – 5 marks*

*Data Structures – 2 marks*

| Location | Label | Opcode | Operand | Object Code |
|----------|-------|--------|---------|-------------|
| 4000     | TEST  | START  | 4000    |             |
| 4000     | FIRST | LDA    | FIVE    | 00400C      |
| 4003     |       | STA    | ALPHA   | 144006      |
| 4006     | ALPHA | RESW   | 2       |             |
| 400C     | FIVE  | WORD   | 5       | 000005      |

#### Data Structures

| SYMTAB |       |
|--------|-------|
| Symbol | Value |
| FIRST  | 4000  |
| ALPHA  | 4006  |
| FIVE   | 400C  |

| OPTAB  |       |
|--------|-------|
| Opcode | Value |
| LDA    | 00    |
| STA    | 14    |

**LOCCTR: 400F**